VECTOR

MICROSAR EEP

Technical Reference

Eep_30_XXspi01

Version 1.03.00

Authors	Mihai Olariu, Michael Goß
Status	Released

Document Information

History

Author	Date	Version	Remarks
Mihai Olariu	2014-10-27	1.00.00	Initial version.
Mihai Olariu	2015-08-04	1.00.01	- Rename the document according to the new naming rule (AR4-R13) - Replace “at least” with “exactly” when define the SPI channel sizes (due to memory optimization and avoid tool validation issues) - Add caution concerning SPI channels order (priority) within the SPI Command and Data Job
Michael Goß	2016-02-09	1.01.00	Update of manual due to new design and high-level/low-level implementation.
Michael Goß	2016-11-08	1.02.00	Additions regarding Hardware Software Interface
Michael Goß	2017-01-09	1.03.00	- Update of manual due to new implementation (No high-level implementation) - Added section 3.1.2.8 Configurable SPI Interface

Reference Documents

No.	Source	Title	Version
[1]	AUTOSAR	AUTOSAR_SWS_EEPROM_Driver.pdf	3.2.0
[2]	AUTOSAR	AUTOSAR_SWS_DevelopmentErrorTracer.pdf	3.2.0
[3]	AUTOSAR	AUTOSAR_SWS_DiagnosticEventManager.pdf	4.2.0
[4]	AUTOSAR	AUTOSAR_SWS_ECUStateManager.pdf	3.0.0
[5]	AUTOSAR	AUTOSAR_SWS_BSWModeManager.pdf	1.2.0
[6]	AUTOSAR	AUTOSAR_SWS_DIODriver.pdf	2.5.0
[7]	AUTOSAR	AUTOSAR_SWS_SPIHandlerDriver.pdf	3.2.0
[8]	AUTOSAR	AUTOSAR_SWS_Mem_AbstractionInterface.pdf	1.4.0
[9]	AUTOSAR	AUTOSAR_SWS_EEPROMAbstraction.pdf	2.0.0
[10]	AUTOSAR	AUTOSAR_TR_BSWModuleList.pdf	1.6.0
[11]	Microchip	Microchip_25LC1024.pdf	05/24/2010

Scope of the document:

This technical reference describes the general use of an external EEPROM which is behaviorally/ opcode compatible with Microchip 25LC1024, which is considered as a reference for the entire class of Spi EEPROM devices. All aspects which are EEPROM cell specific are described in the according manual of Microchip 25LC1024 (see [11]).

Caution
We have configured the programs in accordance with your specifications in the questionnaire. Whereas the programs do support other configurations than the one specified in your questionnaire, Vector´s release of the programs delivered to your company is expressly restricted to the configuration you have specified in the questionnaire.

Contents

Illustrations

Tables

1 Component History

The component history gives an overview over the important milestones that are supported in the different versions of the component.

Component Version	New Features
1.00.00	Initial implementation of ASR4.0.3; anyway the component can be used also in an ASR3.x environment/ project.
1.00.01	Fix some compiler specific warnings.
1.01.00	Update due to new design and high-level/low-level implementation.
1.02.00	Additions regarding Hardware Software Interface
1.03.00	Update due to new implementation

Table 1-1 Component history

2 Introduction

This document describes the functionality, API and configuration of the AUTOSAR BSW module EEP as specified in [1].

Supported AUTOSAR Release*:	4	
Supported Configuration Variants:	pre-compile/ post-build	
Vendor ID:	EEP_30_XXSPI01_VENDOR_ID	30 decimal (= Vector-Informatik, according to HIS)
Module ID:	EEP_30_XXSPI01_MODULE_ID	90 decimal (according to ref. [10])

* For the precise AUTOSAR Release 3.x please see the release specific documentation.

This EEP driver provides services to access non-volatile memory on an external EEPROM device connected via the controller's SPI interface. These services are: data reading, writing, erasing, comparing, communication test and EEPROM write protection unlock.

Please note that the supported EEPROM chips have the same architecture and are behaviorally/ opcode compatible with Microchip 25LC1024, which is used as reference device. For a list of officially supported EEPROMs, please check the EEP pre-configuration file (Eep_30_XXspi01_pre_bswmd.arxml).

In addition to the supported EEPROM chips, this EEP driver works also for FRAM chips – e.g. Ramtron FM25640. The functional operation of a FRAM is similar and opcode compatible to serial EEPROMs. The major difference between a FRAM and a serial EEPROM with the same pinout relates to its superior write performance.

Differences between a FRAM and a standard serial EEPROM:

- the bit 0 in the Status Register (EEPROM busy flag) is unnecessary as the FRAM writes in real-time and is never busy

- no page register is needed and any number of sequential writes may be performed; if the last address of 1FFFh is reached, the counter will roll over to 0000h

Caution
As a consequence of last difference between EEPROM and FRAM, it results that the BSWMD parameter `EepPageSize` (for Ramtron FM25640) can be as big as the chip size (so basically there is no constraint for its value). Please be aware that the SPI load depends directly on this page size parameter value!

2.1 Architecture Overview

The following figure shows where the EEP is located in the AUTOSAR architecture.

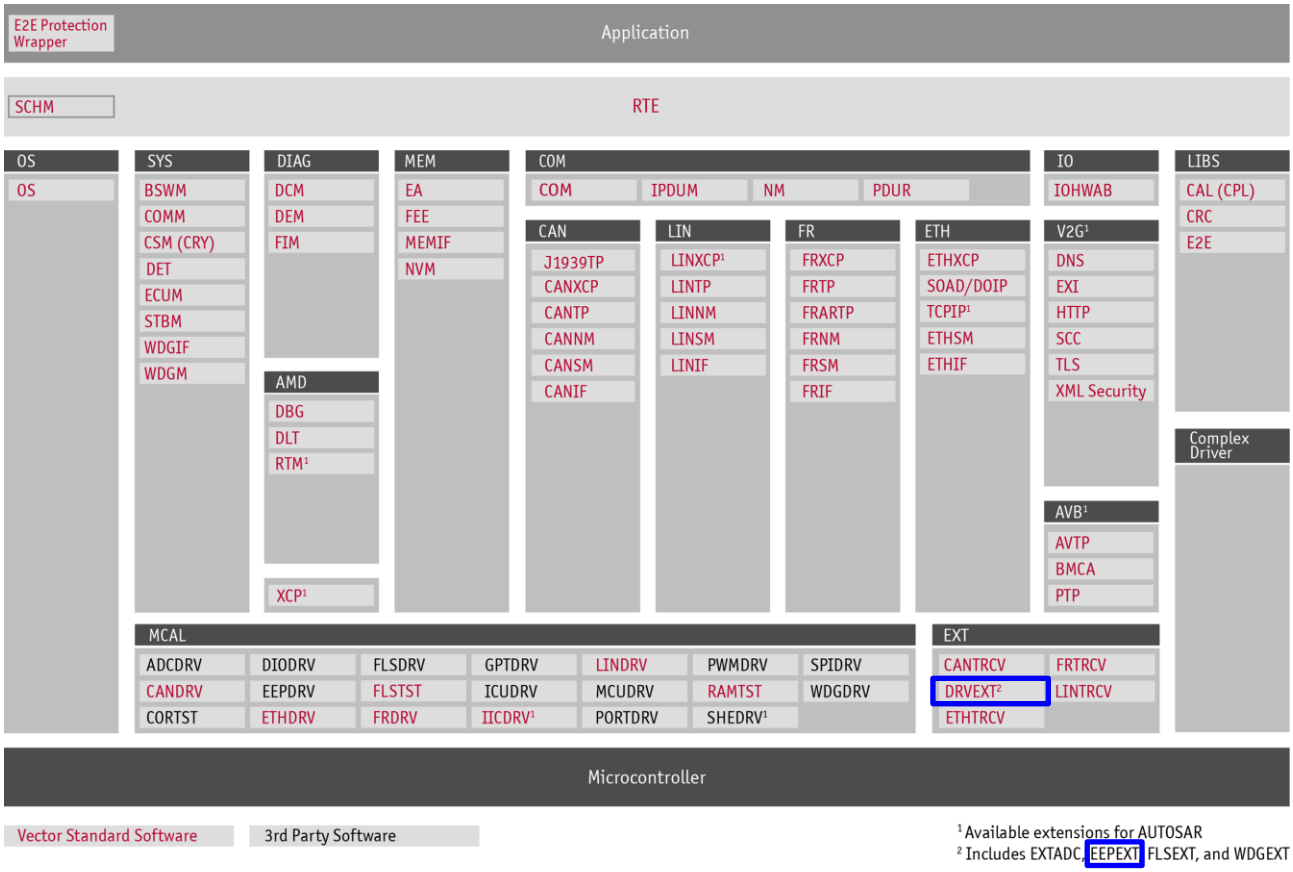

Figure 2-1 AUTOSAR 4.x Architecture Overview

The software architecture is made for two classes of EEPROM drivers:

- EEP modules for on-chip EEPROM: these are part of the Microcontroller Abstraction Layer.
- EEP modules for external EEPROM devices: these are part of the ECU Abstraction Layer.

The current document considers only the case of EEP modules for external EEPROM devices (second case).

The next figure shows the interfaces to adjacent modules of the EEP. These interfaces are described in chapter 5.

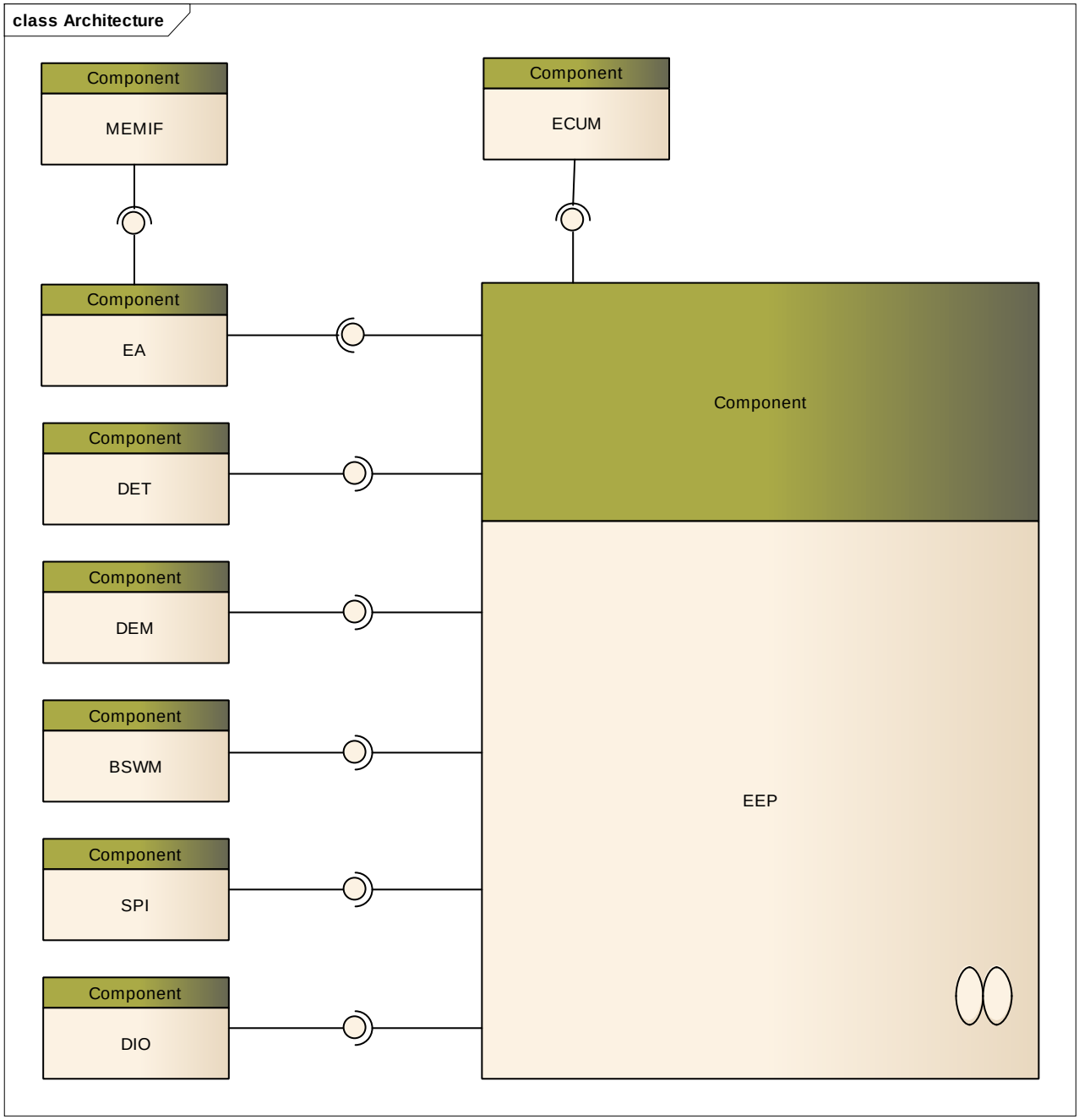

Figure 2-2 Interfaces to adjacent modules of the EEP

Applications do not access the services of the BSW modules directly. They use the service ports provided by the BSW modules via the RTE. The EEP does not provide any service ports.

3 Functional Description

3.1 Features

The features listed in the following tables cover the complete functionality specified for the EEP.

The AUTOSAR standard functionality is specified in [1]. The specified features that are not supported are listed in the table

> Table 3-1 Not supported AUTOSAR standard conform features

Vector Informatik provides further EEP functionality beyond the AUTOSAR standard. The corresponding features are listed in the table

> Table 3-2 Features provided beyond the AUTOSAR standard

Depending on the platform and external EEPROM cell capabilities some features are directly affected and limited. The corresponding features are listed in the table

> Table 3-3 Known Issues and Limitations

3.1.1 Deviations

The following features specified in [1] are not supported:

Supported AUTOSAR Standard Conform Features
Internal EEPROM requirements (e.g. interrupt or polling processing) are not considered
`Eep_GetVersionInfo()` API is not provided as macro
The timeout supervision is not supported for read and compare services
There are not specialized SPI channels for slow and fast mode, but only one set of channels independent of current mode
EEP does not erase a memory cell before writing into it
EEP does not perform read – modify – write operations to preserve data of affected EEPROM cells
EEP does not take into account the write cycle reduction recommendation
Variables to be debugged are not listed in the BSWMD file
Optimized pre-compile time configuration variant is not supported
Optimized link-time configuration variant is not supported

Table 3-1 Not supported AUTOSAR standard conform features

3.1.2 Additions/ Extensions

The following features are provided beyond the AUTOSAR standard:

Features Provided Beyond The AUTOSAR Standard
Multi-channel (indexed APIs) support of up to two EEPROMs
Incremental registers support
Set handling API support
Test communication API support
Unlock EEPROM API support
Disable internal EEPROM write enable latch before processing read/ compare jobs
AUTOSAR 3.x compatibility
Configurable SPI interface

Table 3-2 Features provided beyond the AUTOSAR standard

Info
The module EEP does not buffer data to be read or written. However for compare operations a (configurable) buffer is needed to buffer the results of single SPI transactions.

3.1.2.1 Multi-Channel Support

If the multi-channel feature is activated: the EEP module provides also services for a second physical EEPROM chip.

In this case all the services (inclusively initialization, main function and callbacks) are replicated according to the following rule for the read service:

-> `Eep_30_XXspi01_Read()` - service corresponding to the first instance (channel)

-> `Eep_30_XXspi01_Inst2_Read()` - service corresponding to the second instance (channel)

This feature is automatically activated if configuration contains two EEP modules.

3.1.2.2 Incremental Registers Support

If the EEPROM chip contains an incremental registers area (e.g. ST Microelectronics M35SW16) and the feature is activated, the EEP module provides also a reduced set of APIs for this special incremental registers area partition. The reduced set of APIs contains the following services:

-> `Eep_30_XXspi01_ReadIncReg()`

-> `Eep_30_XXspi01_WriteIncReg()`

-> `Eep_30_XXspi01_CompareIncReg()`

a) Please note that these services behave as defined by the AUTOSAR standard with the constraint that addresses are word aligned.

b) These extra APIs are not shown in the “Module API” tab of the generation tool, but they are active and used by an EA extension module as long as the feature is activated.

As a consequence of 3.1.2.1, the occurrence of the term read service means:

-> `Eep_30_XXspi01_Read()` if multi-channel feature is disabled and incremental registers support feature is disabled

-> `Eep_30_XXspi01_Read()`/ `Eep_30_XXspi01_Inst2_Read()` if multi-channel feature is enabled and incremental registers support feature is disabled

-> `Eep_30_XXspi01_Read()`/ `Eep_30_XXspi01_ReadIncReg()` if multi-channel feature is disabled and incremental registers support feature is enabled

-> `Eep_30_XXspi01_Read()`/`Eep_30_XXspi01_Inst2_Read ()`/ `Eep_30_XXspi01_ReadIncReg()`/`Eep_30_XXspi01_Inst2_ReadIncReg()` if multi-channel feature is enabled and incremental registers support feature is enabled

3.1.2.3 Set Handling API Support

According to the AUTOSAR standard, the EEP main function shall be always called with a fixed recurrence, so that the EEP module would support only one handling mode (a so-called recurrent handling). But there are several scenarios when the application might expect the EEP main function to be called in a “while loop” (a so-called burst handling).

In order to be able to set the current handling during run-time, the EEP module provides a specialized service - `Eep_30_XXspi01_SetHandling()`.

As a consequence of 3.1.2.1, the occurrence of the term set handling service means:

-> `Eep_30_XXspi01_SetHandling()` if multi-channel feature is disabled

-> `Eep_30_XXspi01_SetHandling()`/ `Eep_30_XXspi01_Inst2_SetHandling()` if multi-channel feature is enabled

3.1.2.4 Test Communication API Support

In order to be able to check the EEPROM chip presence and the correct communication, the EEP module provides a specialized service - `Eep_30_XXspi01_TestCom()`.

As a consequence of 3.1.2.1, the occurrence of the term test communication service means:

-> `Eep_30_XXspi01_TestCom()` if multi-channel feature is disabled

-> `Eep_30_XXspi01_TestCom()`/ `Eep_30_XXspi01_Inst2_TestCom()` if multi-channel feature is enabled

3.1.2.5 Unlock EEPROM API Support

In order to be able to unlock (enable the write operation) the EEPROM chip, the EEP module provides a specialized service - `Eep_30_XXspi01_Unlock()`.

As a consequence of 3.1.2.1, the occurrence of the term unlock service means:

-> `Eep_30_XXspi01_Unlock()` if multi-channel feature is disabled

-> `Eep_30_XXspi01_Unlock()`/ `Eep_30_XXspi01_Inst2_Unlock()` if multi-channel feature is enabled

3.1.2.6 Disable Write Enable Latch Before Read/ Compare Processing

If the Reset Latch feature is activated: the EEP module resets the write enable latch of the EEPROM before processing any read/ compare operation.

There are some scenarios when a write command could fail and consequently the write enable latch can remain enabled. As in some strong disturbed environments a READ command can be misinterpreted as WRITE, there can result an unintentional EEPROM writing. In order to prevent something like this, during each EEPROM read sequence, an extra WRDI command will be sent in order to reset the write enable latch.

3.1.2.7 AUTOSAR 3.x Compatibility

Even though this EEP module is implemented according to ASR4.0.3 SWS, it can be used also in an ASR3.x environment/ project according to the Platform_Types.h version. That means:

a) The embedded implementation takes into account the ASR3 compatibility aspects (e.g. critical section signature is different depending on ASR specification).

b) The DaVinci CFG4 tool can be used but only as configurator. Independent of the ASR environment/ project version (ASR3 or ASR4), the EEP module will use the same generator which is a DaVinci CFG5 native generator.

c) The MemMap.h has to be properly prepared to support the ASR3 sections format (e.g. the ASR3 section EEP_30_XXSPI01_START_SEC_CONST_32BIT corresponds to the ASR4 section EEP_30_XXSPI01_START_SEC_CONST_32).

3.1.2.8 Configurable SPI Interface

By default, EEP assumes the SPI interface to apply to the name scheme “Spi*”, i.e. “Spi.h” for the include file or e.g. “Spi_SetupEB(..)” for any Spi service.

If the signature of the used Spi driver differs from this default value, the optional configuration container Eep/EepInitConfiguration/EepSpiInterface can be created in order to manually set the names of all necessary Spi dependencies.

As long as this container exists, the contained names are used in implementation, regardless their actual value. Thus the user has to make sure that the service names and the include file are announced correctly.

3.1.3 Limitations

The following features are limited/ affected under certain circumstances:

Not Supported AUTOSAR Standard Conform Features
EEP does not provide a mechanism for ensuring data integrity
The timeout supervision does not monitor the communication time, but the internal write cycles

Table 3-3 Known Issues and Limitations

3.1.3.1 Data Integrity

The EEP module does not provide mechanisms for ensuring data integrity (for example checksums, redundant storage, etc.). Thus the EEP module can be used within safety relevant systems only if the upper layer software provides one of the above mechanisms for the safety related data.

For the purpose of verification of written data, the compare function of EEP module can be used.

3.1.3.2 Timeout Supervision Based On Internal Write Cycle Monitoring

According to the AUTOSAR standard, the EEP timeouts for all jobs depend on communication properties. For example in case of a write job, the timeout is a function of: current mode, the configured number of bytes to be written in this mode, the size of a EEPROM write data unit and last the maximum time to write one data unit.

This is a tedious formula which would make sense only for a synchronous and uninterrupted communication. Consequently the EEP timeouts are monitored only for write and erase jobs; they use as reference the internal write cycle period which is mentioned in any EEPROM data sheet.

Thus, the timeout in this case means the number of requests the EEP needs to send to the external EEPROM device until it answers with status “ready” (EEPROM device is ready to accept commands again). These requests are asynchronously sent from main function. The response is evaluated in one of the subsequent calls to main function (after the SPI sequence has finished) and if this response does not show that the EEPROM device is ready again, the timeout counter is decremented and a new request is sent. This process is repeated until the EEPROM device signals its readiness or the timeout counter exceeds the configured value. In the latter case a DEM error is reported (according to the current pending job) assuming that the communication is broken.

Caution
So the timeout value is a counter which is time independent in case of burst handling (there is no time base for the main function) or time dependent in case of recurrent handling (the time base is the BSWMD parameter `EepJobCallCycle`).

3.2 Initialization

Before EEP module may be used, it is necessary to initialize the underlying SPI driver as well as the PORT and DIO driver with compatible configurations (regarding how the external EEPROM device is wired to the microcontroller). The driver EEP module itself is initialized by calling the init service (`Eep_30_XXspi01_Init()`) with a pointer to a valid configuration as parameter. Secondly, the internal structures have to be initialized via init memory service (`Eep_30_XXspi01_InitMemory()`).

3.3 States

3.3.1 Module States

The EEP module implements the following state machine:

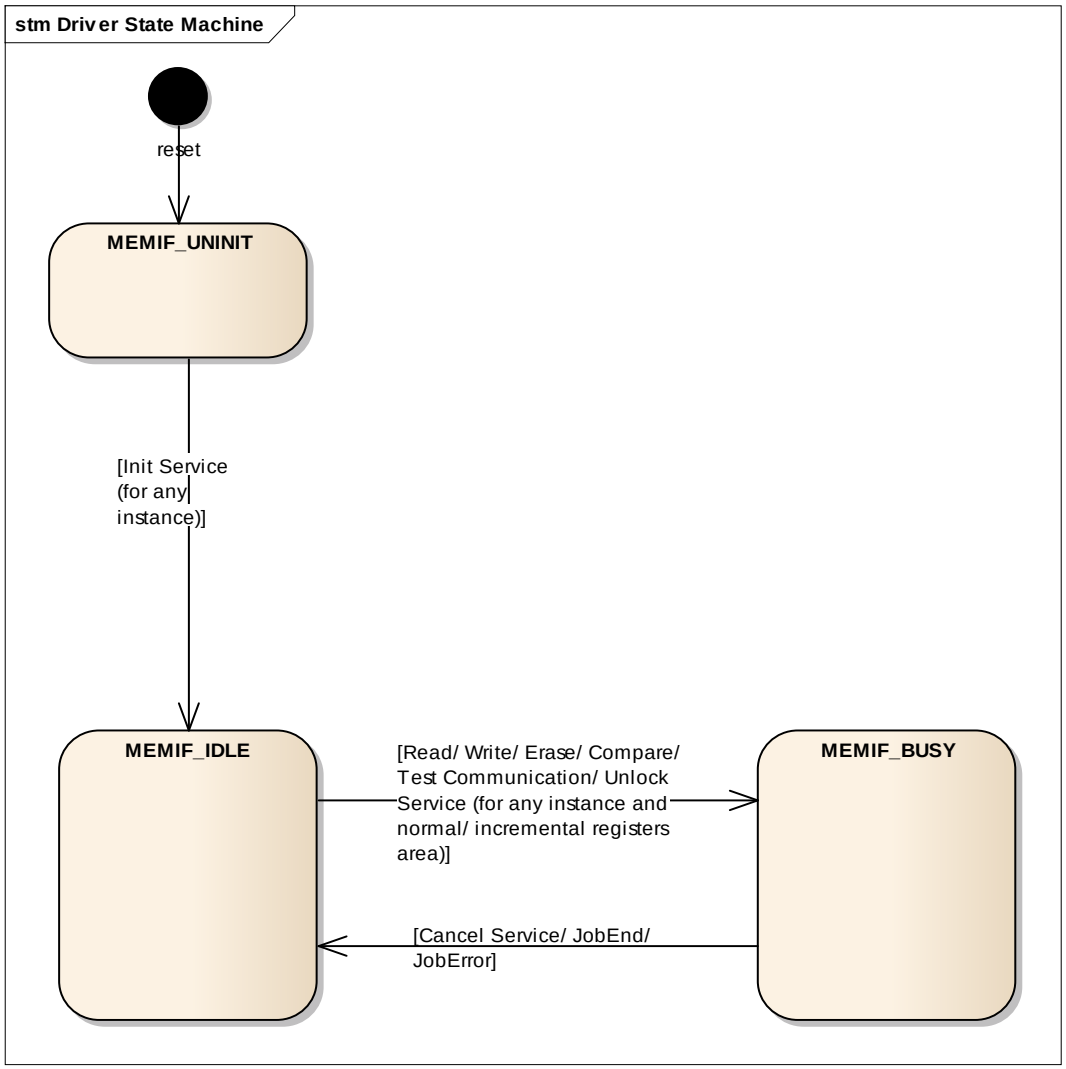

Figure 3-1 Module States

3.3.2 Job States

The EEP module implements the following state machine for job handling:

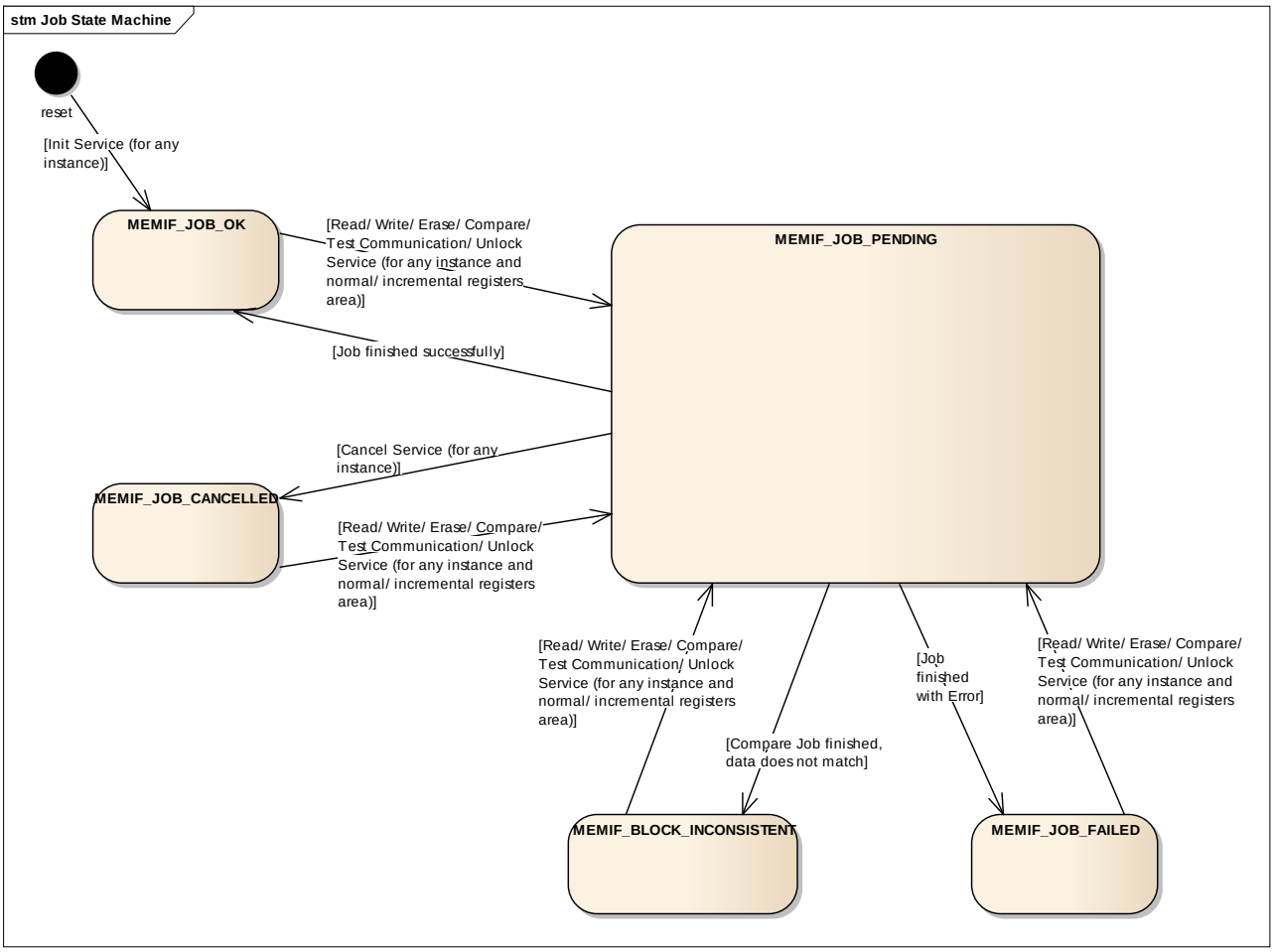

Figure 3-2 Job States

3.4 Main Functions

The EEP has one main function per instance (`Eep_30_XXspi01_MainFunction()` for the first instance) which can be called either with a fixed recurrence or burst depending on the actual handling.

3.4.1 Processing of Jobs

The EEP provides two modes of operation (normal and fast) for read, write, erase and compare jobs. These two modes do not change the behavior of the job processing but provide different data length for processing of jobs. Different data length means here different number of bytes to be transferred over the SPI bus in one transaction.

The test communication and unlock jobs transfer only one byte in any circumstance!

3.4.1.1 Read/ Compare Size

The read and compare jobs have no restriction in the number of bytes to be read (see BSWMD parameters `EepFastReadBlockSize` and `EepNormalReadBlockSize`) in one SPI transaction, so any value up to the size of the EEPROM memory can be configured. This could result in reading the whole EEPROM memory within two main function cycles (that is one SPI transaction).

3.4.1.2 Write/ Erase Size

The write and erase jobs are allowed to transfer in one SPI transaction a maximum amount of bytes equal with the EEPROM hardware page size (see BSWMD parameters `EepFastWriteBlockSize` and `EepNormalWriteBlockSize`), so any value up to the size of the EEPROM page size can be configured. This could result in writing the whole EEPROM page within two main function cycles (that is one SPI transaction).

3.4.2 Finishing and Aborting of Jobs

If any job is successfully completed, the EEP module sets the state to `MEMIF_IDLE` and the job result to `MEMIF_JOB_OK`. When any job encounters a communication issue, the EEP module sets the state to `MEMIF_IDLE` and the job result to `MEMIF_JOB_FAILED`. When a compare job detects data mismatch, the EEP module sets the state to `MEMIF_IDLE` and the job result to `MEMIF_BLOCK_INCONSISTENT`. If the cancel service is called during any job processing, the EEP module sets the state to `MEMIF_IDLE` and the job result to `MEMIF_JOB_CANCELLED`.

After a job has finished successfully, the job end notification callback is executed; respectively after a job has finished erroneously or was canceled, the job error notification callback is executed. These callbacks can be configured to the user's needs, and if there's no need, they can simply be replaced by a `NULL_PTR`. This indicates that a notification shall not be performed.

Caution

The cancel service will abort any type of EEP job immediately, so a new job can be started right after the function has been processed. Any new request is accepted after calling the cancel service, however the pending job and its SPI transaction(s) will be executed until the EEPROM is ready again (this can be immediately in case of a pending read job or after some milliseconds in case of a pending write job with internal write cycle active). This may cause an unavoidable delay for the new job being processed.

In case a job has been aborted by means of cancel service, there is no guarantee that data written to memory during the aborted job is correct!

3.5 Error Handling

3.5.1 Development Error Reporting

By default, development errors are reported to the DET using the service `Det_ReportError()` as specified in [2], if development error reporting is enabled (see BSWMD parameter `EepDevErrorDetect`).

The reported EEP ID is **90**.

The following table presents the reported service IDs and the related services:

Service ID	Service
EEP_30_XXSPI01_SID_INIT	Init service
EEP_30_XXSPI01_SID_SET_MODE	SetMode service
EEP_30_XXSPI01_SID_READ	Read service
EEP_30_XXSPI01_SID_WRITE	Write service
EEP_30_XXSPI01_SID_ERASE	Erase service
EEP_30_XXSPI01_SID_COMPARE	Compare service
EEP_30_XXSPI01_SID_CANCEL	Cancel service
EEP_30_XXSPI01_SID_GET_STATUS	GetStatus service
EEP_30_XXSPI01_SID_GET_JOB_RESULT	GetJobResult service
EEP_30_XXSPI01_SID_MAIN_FUNCTION	Main function
EEP_30_XXSPI01_SID_GET_VERSION_INFO	GetVersionInfo service
EEP_30_XXSPI01_SID_COM_END	ComEnd callback
EEP_30_XXSPI01_SID_SET_HANDLING	SetHandling service
EEP_30_XXSPI01_SID_TEST_COM	TestCommunication service
EEP_30_XXSPI01_SID_SET_LOCK_LEVEL	Unlock service

Table 3-4 Service IDs

The errors reported to DET are described in the following table:

Error Code	Description
EEP_30_XXSPI01_E_PARAM_CONFIG	API service called with wrong parameter
EEP_30_XXSPI01_E_PARAM_ADDRESS	API service called with wrong parameter
EEP_30_XXSPI01_E_PARAM_DATA	API service called with wrong parameter
EEP_30_XXSPI01_E_PARAM_LENGTH	API service called with wrong parameter
EEP_30_XXSPI01_E_UNINIT	API service called without module initialization
EEP_30_XXSPI01_E_BUSY	API service called while driver still busy
EEP_30_XXSPI01_E_TIMEOUT	Timeout exceeded
EEP_30_XXSPI01_E_PARAM_POINTER	API service called with a NULL pointer

Table 3-5 Errors reported to DET

3.5.1.1 State and Parameter Checking

AUTOSAR requires that API functions check the validity of component status, function parameters, etc. The checks in Table 3-6 are internal parameter checks of the listed API functions. These checks are for development error reporting and can be en-/disabled via the parameter `EepDevErrorDetect`.

The following table shows which parameter checks are performed on which services:

Service \ Check	EEP_30_XXSPI01_E_PARAM_CONFIG	EEP_30_XXSPI01_E_PARAM_ADDRESS	EEP_30_XXSPI01_E_PARAM_DATA	EEP_30_XXSPI01_E_PARAM_LENGTH	EEP_30_XXSPI01_E_UNINIT	EEP_30_XXSPI01_E_BUSY	EEP_30_XXSPI01_E_TIMEOUT	EEP_30_XXSPI01_E_PARAM_POINTER
Init service	■							
SetMode service					■	■		
Read service		■	■	■	■	■		
Write service		■	■	■	■	■		
Erase service		■		■	■	■		
Compare service		■	■	■	■	■		
Cancel service					■			
GetStatus service								
GetJobResult service					■			
Main function							■	
GetVersionInfo service								■
ComEnd callback								
SetMode service					■	■		
SetHandling service					■	■		
TestCommunication service					■	■		
Unlock service					■	■		

Table 3-6 Development Error Reporting: Assignment of checks to services

3.5.2 Production Code Error Reporting

Production code related errors are reported to DEM using the service `Dem_ReportErrorStatus()` as specified in [3]. Production errors are hardware errors and software exceptions that cannot be avoided and therefore the DEM is notified if they occur.

If another module is used for production error reporting, the function prototype for reporting the error can be configured by the integrator, but must have the same signature as the service `Dem_ReportErrorStatus()`.

The errors reported to DEM are described in the following table:

Error Code	Description
`EEP_E_READ_FAILED`	EEPROM read failed (HW)
`EEP_E_COMPARE_FAILED`	EEPROM compare failed (HW)
`EEP_E_TEST_COM_FAILED`	EEPROM test communication failed (HW)
`EEP_E_WRITE_FAILED`	EEPROM write failed (HW)
`EEP_E_ERASE_FAILED`	EEPROM erase failed (HW)
`EEP_E_SET_LOCK_LEVEL_FAILED`	EEPROM unlock failed (HW)

Table 3-7 Errors reported to DEM

3.5.2.1 Hardware and Software Exceptions Checking

AUTOSAR requires that some API functions check some production error scenarios. The checks in Table 3-8 are hardware/ software exceptions specific checks within the listed API functions. These checks are for production code error reporting and cannot be disabled.

The following table shows which parameter checks are performed on which services:

Service \ Check	EEPROM read failed (HW)	EEPROM compare failed (HW)	EEPROM test communication failed (HW)	EEPROM write failed (HW)	EEPROM erase failed (HW)	EEPROM unlock failed (HW)
Main function	■	■	■	■	■	■

Table 3-8 Production Code Error Reporting: Assignment of checks to services

3.6 Hardware Software Interface

The EEP uses a set of instructions in order to access the EEPROM device via SPI bus. Table 3-9 contains a list of instruction bytes for device operation which are used by EEP driver.

Instruction Name	Instruction Format	Description
READ	0000 0011	Read data from memory array
WRITE	0000 0010	Write data to memory array
WREN	0000 0110	Set write enable latch (enable write operations)
WRDI	0000 0100	Reset write enable latch (disable write operations)
RDSR	0000 0101	Read Status Register
WRSR	0000 0001	Write Status Register
WRINC	0000 0111	Write data to incremental registers

Table 3-9 Used EEPROM Instruction Set

If an EEPROM device does not provide incremental registers, the command WRINC will not be used by the EEP. Availability of incremental registers needs to be configured via EepDeviceInformation/EepIncRegSupport switch in configuration.

4 Integration

This chapter gives necessary information for the integration of the MICROSAR EEP into an application environment of an ECU.

4.1 Scope of Delivery

The delivery of the EEP contains the files which are described in the chapters 4.1.1 and 4.1.2:

4.1.1 Static Files

File Name	Description
Eep_30_XXspi01_bswmd.arxml	The description file contains the formal notation of all configuration parameters of the EEP.
Eep_30_XXspi01_pre_bswmd.arxml	The pre-configuration file contains the necessary hardware specific values, e.g. list of supported EEPROM devices.
Eep_30_XXspi01_SafeBSW_pre.arxml	The recommended configuration sets parameters which are necessary within a SafeBSW environment. This file will only be delivered in SafeBSW projects.
Identifier.xml	This file lists and defines all configuration parameters that the DaVinci CFG4 needs. This file is relevant only if the EEP module is used within an ASR3 project/ environment when parameterization is performed via CFG4.
DrvEep_XXspi01Asr.jar	This file is the CFG5 generator.
Eep_30_XXspi01.c	This file holds the definitions of the public API services.
Eep_30_XXspi01.h	This is a common include file. It represents the module’s common declarations.
Eep_30_XXspi01_Proc.c	This file holds the major part of the implementation and processing functions.
Eep_30_XXspi01_Proc.h	This header file publishes the job processing functionality.
Eep_30_XXspi01_Cbk.h	This defines the callback interface to the EEP.
Eep_30_XXspi01_cfg.mak	This file contains the configurable part of the Makefile.
Eep_30_XXspi01_check.mak	This file implements the Makefile checks.
Eep_30_XXspi01_defs.mak	This file contains the Makefile definitions.
Eep_30_XXspi01_rules.mak	This file implements the Makefile rules.

Table 4-1 Static files

4.1.2 Dynamic Files

The dynamic files are generated by the DaVinci CFG5 configuration tool.

File Name	Description
Eep_30_XXspi01_PBcfg.c	Contains the generated post-build data.
Eep_30_XXspi01_Lcfg.c	Contains generated link-time data.
Eep_30_XXspi01_Cfg.h	Contains generated public pre-compile data.

Table 4-2 Generated files

4.2 Include Structure

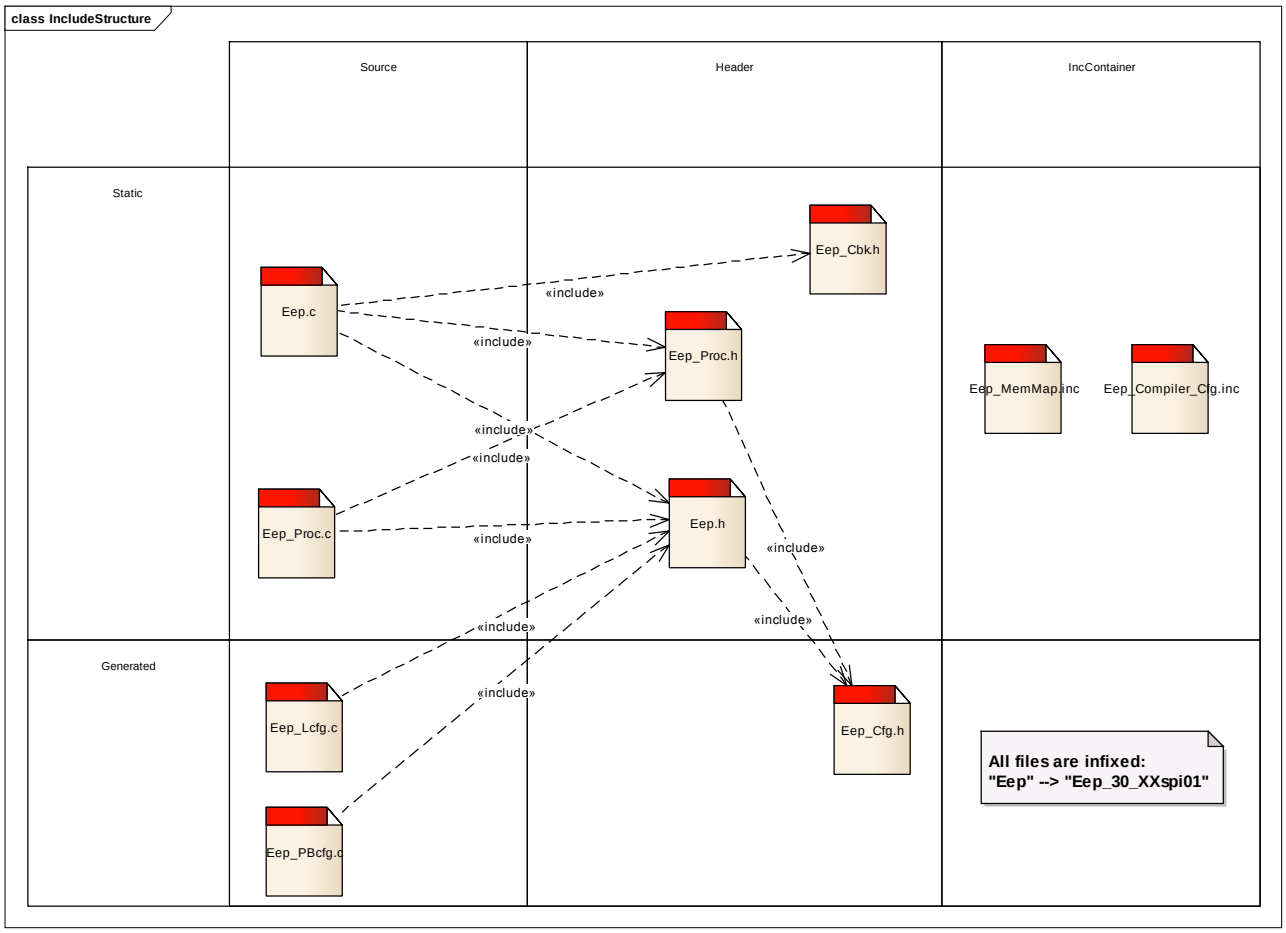

Figure 4-1 Include Structure

4.3 Compiler Abstraction and Memory Mapping

The objects (e.g. variables, functions, constants) are declared by compiler independent definitions – the compiler abstraction definitions. Each compiler abstraction definition is assigned to a memory section.

The following table contains the memory section names and the compiler abstraction definitions defined for the EEP and illustrate their assignment among each other.

Memory Mapping Sections \ Compiler Abstraction Definitions	EEP_30_XXSPI01_CODE	EEP_30_XXSPI01_APPL_CODE	EEP_30_XXSPI01_CONST	EEP_30_XXSPI01_PBCFG	EEP_30_XXSPI01_VAR_NO_INIT	EEP_30_XXSPI01_VAR_INIT	EEP_30_XXSPI01_APPL_DATA
EEP_30_XXSPI01_START_SEC_CODE EEP_30_XXSPI01_STOP_SEC_CODE	■						
EEP_30_XXSPI01_START_SEC_CONST_32 EEP_30_XXSPI01_STOP_SEC_CONST_32			■				
EEP_30_XXSPI01_START_SEC_CONST_UNSPECIFIED EEP_30_XXSPI01_STOP_SEC_CONST_UNSPECIFIED			■				
EEP_30_XXSPI01_START_SEC_PBCFG_ROOT EEP_30_XXSPI01_STOP_SEC_PBCFG_ROOT				■			
EEP_30_XXSPI01_INST2_START_SEC_PBCFG_ROOT EEP_30_XXSPI01_INST2_STOP_SEC_PBCFG_ROOT				■			
EEP_30_XXSPI01_START_SEC_VAR_NO_INIT_UNSPECIFIED EEP_30_XXSPI01_STOP_SEC_VAR_NO_INIT_UNSPECIFIED					■		
EEP_30_XXSPI01_START_SEC_VAR_INIT_UNSPECIFIED EEP_30_XXSPI01_STOP_SEC_VAR_INIT_UNSPECIFIED						■	

Table 4-3 Compiler abstraction and memory mapping

4.4 Critical Sections

The AUTOSAR standard provides within the BSW Mode Manager a BSW Scheduler, which handles entering and leaving critical sections. For more information about the BSW Scheduler please refer to [5]. The EEP provides a critical section that has to be mapped by the BSW Scheduler according to description:

Critical Section	Description
EEP_30_XXSPI01_EXCLUSIVE_AREA_0	Requires blocking of the global interrupt in order to ensure data consistency during job processing/ cancelation.

Table 4-4 EEP Critical sections

4.5 Dependencies on SW modules

4.5.1 DET (optional)

The Development Error Tracer (see [2]) records all development errors. The EEP reports all development errors to DET. The detection and reporting of development errors is controlled by the BSWMD parameter `EepDevErrorDetect`.

4.5.2 DEM

The Diagnostic Event Manager (see [3]) records all production errors for diagnostic purposes. The EEP reports all production errors to DEM. The detection and reporting of production errors cannot be switched off.

4.5.3 ECUM (optional)

The ECU State Manager (see [4]) is normally responsible for components initialization.

4.5.4 BSWM

The BSW Mode Manager (see [5]) is responsible for allot of functionalities, but concerning EEP it implements its appropriate critical section. For details regarding the EEP critical section please check chapter 4.4.

4.5.5 DIO (optional)

The digital input/ output (see [6]) provides the abstract channel to control I/ O pins. If the EEP is configured to support the unlock service, then one DIO channel is used (see BSWMD parameter `EepDioReference`).

4.5.6 SPI

The SPI driver (see [7]) provides the communication entities (sequences, jobs and channels) used to communicate with the external EEPROM device via the microcontroller's SPI interface. The dependency with SPI module is realized over the BSWMD parameter `EepSpiReference`.

4.5.7 MEMIF

The memory interface (see [8]) abstracts the upper software layers to an instance of EA module. It too provides some data types directly to the driver, e.g. the `MemIf_StatusType`.

4.5.8 EA

Module EA (see [9]) provides uniform access from upper layers to the EEPROM memory of underlying drivers.

5 API Description

For an interfaces overview please see Figure 2-2.

5.1 Type Definitions

The types defined by the EEP are described in this chapter.

Type Name	C-Type	Description	Value Range
Eep_30_XXspi01_HandlingType	enum	This is the EEP handling type, i.e. the way the EEP may function is called.	EEP_30_XXEEP01_HANDLING_BURST
			EEP_30_XXEEP01_HANDLING_RECURRENT
Eep_30_XXspi01_LockLevelType	Enum	This is the EEP write protection level, which corresponds to EEPROM's block protection bits in status register.	0x00, 0x04, 0x08, 0x0C, 0x80, 0x84, 0x88, 0x8C
Eep_30_XXspi01_AddressType	uint16/ uint32	This is the EEP minimum address width. If the EEP used size > 64 Kbytes it has to be uint32.	0x0000 … 0xFFFF respectively 0x00000000 – 0xFFFFFFFF
Eep_30_XXspi01_LengthType	uint16/ uint32	This is the EEP minimum length width. If the EEP used size > 64 Kbytes it has to be uint32.	0x0000 … 0xFFFF respectively 0x00000000 – 0xFFFFFFFF
Eep_30_XXspi01_NotificationType	func	This is the notification callback type for job end and job error notifications	symbolic name
Eep_30_XXspi01_ChipType	struct	This type contains the EEPROM chip properties (e.g. page size, write time, etc.). Concerning its embedded members please check the type implementation.	-
Eep_30_XXspi01_ConfigType	struct	This type is the	-

Type Name	C-Type	Description	Value Range
		EEP configuration. Concerning its embedded members please check the type implementation.	

Table 5-1 Type definitions

Caution
Depending on configuration options, some members of `Eep_30_XXspi01_ConfigType` may be absent.

5.2 Interrupt Service Routines provided by EEP

The EEP itself does not implement any interrupt.

5.3 Services provided by EEP

The EEP API consists of services, which are realized by function calls in case of single-channel and multi-channel configuration. For details concerning the specialized APIs that apply to a specific EEPROM instance (first or second) or EEPROM region (memory or incremental registers) please read the chapters 3.1.2.1 and 3.1.2.2.

5.3.1 Init Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameter/ return value and flow apply also for the second instance.

Prototype	
`void` **`Eep_30_XXspi01_Init`** `(const Eep_30_XXspi01_ConfigType *ConfigPtr)`	
Parameter	
`ConfigPtr`	Parameter to a selected configuration structure
Return code	
`void`	--
Functional Description	
Service for driver initialization. This function initializes the module according to the parameter `ConfigPtr`. After having finished the module initialization, the EEP state is set to `MEMIF_IDLE` and the job result is set to `MEMIF_JOB_OK`. The EEP mode is set to the configured default mode (Slow or Fast) and the EEP handling is set to the configured default handling (Burst or Recurrent).	

Particularities and Limitations
■ This service may not be called during a running operation. ■ This function is synchronous. ■ This function is non-reentrant. ■ This function is always available.
Expected Caller Context
■ Expected to be called by the ECU state manager to initialize the EEPROM driver.

Table 5-2 Init Service

5.3.2 InitMemory Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameter/ return value and flow apply also for the second instance.

Prototype	
`void` **`Eep_30_XXspi01_InitMemory`** `(void)`	
Parameter	
`void`	--
Return code	
`void`	--
Functional Description	
Service for initialization of internal structures. This function initializes the Chip configuration and internal buffers of the EEP module.	
Particularities and Limitations	
■ This service may not be called during a running operation. ■ This function is synchronous. ■ This function is non-reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called by the ECU state manager to initialize internal structures of EEPROM driver.	

Table 5-3 InitMemory Service

5.3.3 SetMode Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameter/ return value and flow apply also for the second instance.

Prototype
`void` **`Eep_30_XXspi01_SetMode`** `(MemIf_ModeType Mode)`

Parameter	
Mode	see MemIf_ModeType in document [8]
Return code	
void	--
Functional Description	
Service for EEP mode selection. This service sets the EEP operation mode to the given mode parameter.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ This service may not be called during a running operation. ■ This function is synchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-4 SetMode Service

5.3.4 SetHandling Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameter/ return value and flow apply also for the second instance.

Prototype	
`void` **`Eep_30_XXspi01_SetHandling`**`(Eep_30_XXspi01_HandlingType Handling)`	
Parameter	
`Handling`	see `Eep_30_XXspi01_HandlingType` in chapter 5.1
`Return code`	
`void`	--
Functional Description	
Service for EEP handling selection. This service sets the EEP main function handling, i.e. it will be called either in a burst manner or with a fixed cyclic recurrence.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ This service may not be called during a running operation. ■ This function is synchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-5 SetHandling Service

5.3.5 Read Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameters/ return value and flow apply also for the second instance and incremental registers area partition; the only extra constraint for this incremental partition is that addresses are word aligned.

Prototype	

`Std_ReturnType` **`Eep_30_XXspi01_Read`** `(`
`  Eep_30_XXspi01_AddressType EepromAddress,`
`  uint8 *DataBufferPtr,`
`  Eep_30_XXspi01_LengthType Length)`	

Parameter	
`EepromAddress`	Address in EEPROM Min.: 0 Max.: EEPROM chip memory size - 1
`DataBufferPtr`	Pointer to destination data buffer in RAM
`Length`	Number of bytes to read Min.: 1 Max.: EEPROM chip memory size
Return code	
`Std_ReturnType`	`E_OK`: read command has been accepted `E_NOT_OK`: read command has not been accepted
Functional Description	
Service for reading from EEPROM. This service copies the given parameters, initiates a read job, sets the EEP status to `MEMIF_BUSY`, sets the job result to `MEMIF_JOB_PENDING` and returns. The read job is executed asynchronously within the EEP job processing function (`Eep_30_XXspi01_MainFunction`). During job processing a data block of size `Length` is read from `EepromAddress` to `*DataBufferPtr`.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ Only one EEP job (read/ write/ erase/ compare/ test communication/ unlock) can be accepted at the same time. ■ This function is asynchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-6 Read Service

5.3.6 Write Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameters/ return value and flow apply also for the second instance and incremental registers area partition; the only extra constraint for this incremental partition is that addresses are word aligned.

Prototype	

`Std_ReturnType` **`Eep_30_XXspi01_Write`** `(`
`Eep_30_XXspi01_AddressType EepromAddress,`
`uint8 *DataBufferPtr,`
`Eep_30_XXspi01_LengthType Length)`	

Parameter	
`EepromAddress`	Address in EEPROM Min.: 0 Max.: EEPROM chip memory size - 1
`Length`	Number of bytes to write Min.: 1 Max.: EEPROM chip memory size
`DataBufferPtr`	Pointer to source data
`Return code`	
`Std_ReturnType`	`E_OK`: write command has been accepted `E_NOT_OK`: write command has not been accepted
Functional Description	
Service for writing to EEPROM. This service copies the given parameters, initiates a write job, sets the EEP status to `MEMIF_BUSY`, sets the job result to `MEMIF_JOB_PENDING` and returns. The write job is executed asynchronously within the EEP job processing function (`Eep_30_XXspi01_MainFunction`). During job processing a data block of size `Length` is written from `*DataBufferPtr` to `EepromAddress`.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ Only one EEP job (read/ write/ erase/ compare/ test communication/ unlock) can be accepted at the same time. ■ This function is asynchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-7 Write Service

5.3.7 Erase Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameters/ return value and flow apply also for the second instance.

Prototype	
`Std_ReturnType` **`Eep_30_XXspi01_Erase`** `( Eep_30_XXspi01_AddressType EepromAddress, Eep_30_XXspi01_LengthType Length)`	
Parameter	
`EepromAddress`	Address in EEPROM Min.: 0 Max.: EEPROM chip memory size - 1
`Length`	Number of bytes to erase Min.: 1 Max.: EEPROM chip memory size
`Return code`	
`Std_ReturnType`	`E_OK`: erase command has been accepted `E_NOT_OK`: erase command has not been accepted
Functional Description	
Service for erasing EEPROM sections. This service copies the given parameters, initiates an erase job, sets the EEP status to `MEMIF_BUSY`, sets the job result to `MEMIF_JOB_PENDING` and returns. The erase job is executed asynchronously within the EEP job processing function (`Eep_30_XXspi01_MainFunction`). An EEPROM block starting from `EepromAddress` with size `Length` is erased. Some of the supported EEPROM devices do not have a native erase-command, so erasing is emulated by writing the erase value to the memory locations to be erased. This erase value can be configured according to the application needs as default value of the communication driver (see SPI).	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ Only one EEP job (read/ write/ erase/ compare/ test communication/ unlock) can be accepted at the same time. ■ This function is asynchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-8 Erase Service

5.3.8 Compare Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameters/ return value and flow apply also for the second instance and incremental registers area partition; the only extra constraint for this incremental partition is that addresses are word aligned.

Prototype	
`Std_ReturnType Eep_30_XXspi01_Compare ( Eep_30_XXspi01_AddressType EepromAddress, const uint8 *DataBufferPtr, Eep_30_XXspi01_LengthType Length)`	
Parameter	
`EepromAddress`	Address in EEPROM Min.: 0 Max.: EEPROM chip memory size - 1
`Length`	Number of bytes to compare Min.: 1 Max.: EEPROM chip memory size
`DataBufferPtr`	Pointer to data buffer (compare data)
Return code	
`Std_ReturnType`	`E_OK`: compare command has been accepted `E_NOT_OK`: compare command has not been accepted
Functional Description	
Service for comparing data in the EEPROM device's memory with data in RAM. This service copies the given parameters, initiates a compare job, sets the EEP status to `MEMIF_BUSY`, sets the job result to `MEMIF_JOB_PENDING` and returns. The compare job is executed asynchronously within the EEP job processing function (`Eep_30_XXspi01_MainFunction`). During job processing an EEPROM data block at `EepromAddress` with size `Length` is compared with a data block at `*DataBufferPtr` of the same length.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ Only one EEP job (read/ write/ erase/ compare/ test communication/ unlock) can be accepted at the same time. ■ This function is asynchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-9 Compare Service

5.3.9 TestCom Service

Please note that the table below considers the API for the first instance regarding this functionality. The parameters/ return value and flow apply also for the second instance.

Prototype	
`Std_ReturnType` **`Eep_30_XXspi01_TestCom`** `(void)`	
Parameter	
`void`	--
Return code	
`Std_ReturnType`	`E_OK`: read command has been accepted `E_NOT_OK`: read command has not been accepted
Functional Description	
Service for testing the communication with the EEPROM (device present test). This service initiates a test communication job, sets the EEP status to `MEMIF_BUSY`, sets the job result to `MEMIF_JOB_PENDING` and returns. The test communication job is executed asynchronously within the EEP job processing function (`Eep_30_XXspi01_MainFunction`). During job processing the EEPROM shall report its readiness.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ Only one EEP job (read/ write/ erase/ compare/ test communication/ unlock) can be accepted at the same time. ■ This function is asynchronous. ■ This function is reentrant.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-10 TestCom Service

5.3.10 Unlock Service

Please note that the table below considers the API for the first instance regarding this functionality. The parameters/ return value and flow apply also for the second instance.

Prototype	
`Std_ReturnType` **`Eep_30_XXspi01_Unlock`** `(Eep_30_XXspi01_LockLevelType LockLevel)`	
Parameter	
`LockLevel`	Defines the write protection which is set during the Unlock job
`Return code`	
`Std_ReturnType`	`E_OK`: read command has been accepted `E_NOT_OK`: read command has not been accepted
Functional Description	
Service for unlocking the EEPROM. This service initiates a set lock level job, sets the EEP status to `MEMIF_BUSY`, sets the job result to `MEMIF_JOB_PENDING` and returns. The unlock job is executed asynchronously within the EEP job processing function (`Eep_30_XXspi01_MainFunction`). During job processing the block protection bits of EEPROM's status register are set according to LockLevel. EEPROM needs to support write protection via block protection bits.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ Only one EEP job (read/ write/ erase/ compare/ test communication/ unlock) can be accepted at the same time. ■ This function is asynchronous. ■ This function is reentrant.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-11 Unlock Service

5.3.11 Cancel Service

Please note that the table below considers the API for the first instance regarding this functionality. The parameters/ return value and flow apply also for the second instance.

Prototype	
`void` **`Eep_30_XXspi01_Cancel`** `(void)`	
Parameter	
`void`	--
Return code	
`void`	--
Functional Description	
Service for cancelling a running EEP job (read/ write/ erase/ compare/ test communication/ unlock). This function aborts a running job synchronously so that directly after returning from this function a new job (e.g. write) can be requested by the upper layer. This function resets the module state to `MEMIF_IDLE`. If the job result has the value `MEMIF_JOB_PENDING`, this function sets the job result to `MEMIF_JOB_CANCELLED`. Otherwise it leaves the job result unchanged.	
Particularities and Limitations	
■ The EEPROM driver has to be initialized before this service is called. ■ The states and data of the affected EEPROM cells are undefined in case of cancelling a write/ erase job. ■ This function is synchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-12 Cancel Service

5.3.12 GetStatus Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The return value and flow apply also for the second instance.

Prototype	
`MemIf_StatusType` **`Eep_30_XXspi01_GetStatus`** `(void)`	
Parameter	
`void`	--
`Return code`	
`MemIf_StatusType`	see MemIf_StatusType in document [8]
Functional Description	
This service returns the EEPROM driver status synchronously.	
Particularities and Limitations	
■ This function is synchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-13 GetStatus Service

5.3.13 GetJobResult Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The return value and flow apply also for the second instance.

Prototype	
`MemIf_JobResultType` **`Eep_30_XXspi01_GetJobResult`** `(void)`	
Parameter	
`void`	--
`Return code`	
`MemIf_JobResultType`	see MemIf_JobResultType in document [8]
Functional Description	
This service will return the result of the last job synchronously.	
Particularities and Limitations	
■ The services read/ write/ erase/ compare/ test communication/ unlock share the same job status variable. ■ Only the result of the latest processed job can be queried. Every new job that has been accepted overwrites the job result with `MEMIF_JOB_PENDING`. ■ This function is synchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in the application context.	

Table 5-14 GetJobResult Service

5.3.14 Main Function

Please note that the table below considers the main function for the first instance regarding this functionality (as defined by AUTOSAR standard). The flow applies also for the second instance.

Prototype	
`void` **`Eep_30_XXspi01_MainFunction`** `(void)`	
Parameter	
`void`	--
Return code	
`void`	--
Functional Description	
This is the EEP main function. It performs the processing of the EEP jobs (read/ write/ erase/ compare/ test communication/ unlock). When a job has been initiated, this function has to be called cyclically (according to the current handling type) until the job is finished. This function may also be called cyclically even if no job is pending. In this case the function returns without action.	
Particularities and Limitations	
■ This function is synchronous. ■ This function is reentrant. ■ This function is always available.	
Expected Caller Context	
■ Expected to be called in an OS task context.	

Table 5-15 Main Function

5.3.15 GetVersionInfo Service

Please note that the table below considers the API for the first instance regarding this functionality (as defined by AUTOSAR standard). The parameter/ return value and flow apply also for the second instance.

Prototype	
`void` **`Eep_30_XXspi01_GetVersionInfo`** `(Std_VersionInfoType* versioninfo)`	
Parameter	
`versioninfo`	Data structure to which EEP's version information shall be written.
`Return code`	
`void`	--
Functional Description	
This function returns the version information of module EEP. The input/ output parameter versioninfo is filled with the version information of the module. The service can be en-/ disabled according to BSWMD parameter `EepVersionInfoApi`.	
Particularities and Limitations	
■ This function is synchronous. ■ This function is reentrant.	
Expected Caller Context	
■ Expected to be called in any context.	

Table 5-16 GetVersionInfo Service

5.4 Services used by EEP

In the following table are listed services provided by other components, which are used by the EEP for different purposes. For details about prototype and functionality refer to the providing component.

Component	API
DET	`Det_ReportError()`
DEM	`Dem_ReportErrorStatus()`
BSWM	`SchM_Enter_Eep_30_XXspi01_EEP_30_XXSPI01_EXCLUSIVE_AREA_0()` `SchM_Exit_Eep_30_XXspi01_EEP_30_XXSPI01_EXCLUSIVE_AREA_0()`
DIO	`Dio_ReadChannel()` `Dio_WriteChannel()`

SPI	`Spi_SetupEB()`
	`Spi_AsyncTransmit()`
	`Spi_GetSequenceResult()`

Table 5-17 Services used by the EEP

5.5 Callback Functions

This chapter describes the callback functions that are implemented by the EEP and can be invoked by other modules. The prototypes of the callback functions are provided in the header file `Eep_30_XXspi01_Cbk.h` by the EEP.

5.5.1 Communication End Callback

The communication between the external EEPROM device and module EEP is performed asynchronously. For synchronization of the two drivers (SPI and EEP) a callback interface is provided by EEP. In the configuration of module SPI the header file Eep_30_XXspi01_Cbk.h has to be included and the communication end callback function has to be configured as sequence end notification for all the SPI sequences used by EEP.

Please note that the table below considers the callback for the first instance regarding this functionality (as defined by AUTOSAR standard). The flow applies also for the second instance.

Prototype	
`void` **`Eep_30_XXspi01_ComEnd`** `(void)`	
Parameter	
`void`	--
Return code	
`void`	--
Functional Description	
This function has to be called by the communication driver to synchronize communication and job processing. The call occurs on completion of each SPI sequence that is used by EEP.	
Particularities and Limitations	
■ This function is always available.	
Expected Caller Context	
■ Expected to be called in any context (see how the communication driver implements the sequence-end notification).	

Table 5-18 Communication End Callback

5.6 Configurable Interfaces

5.6.1 Notifications

At its configurable interfaces the EEP defines notifications that can be mapped to callback functions provided by other modules. The mapping is performed at configuration time (see BSWMD parameters `EepJobEndNotification` and `EepJobErrorNotification`); the function prototypes have to match the appropriate void <`func`> (void) signatures.

Component	API
EA	`Ea_JobEndNotification()` `Ea_JobErrorNotification()`

Table 5-19 Notifications used by the EEP

Caution
Do not configure these notifications in case that the support for incremental registers is activated. Background: there is no information for the upper layer to conclude from which partition the notification came from (either normal area or incremental registers area partition)!!!

Caution
Do not configure these notifications in case that the support for test communication service or unlock service is activated. Background: these are extension services that are normally not used by EA.

5.6.2 Callout Functions

The EEP does not provide any callout function.

5.6.3 Hook Functions

The EEP does not provide any hook function.

5.7 Service Ports

The EEP does not provide any service port.

6 Configuration

6.1 Configuration Variants

The EEP only supports the configuration variant

> VARIANT-PRE-COMPILE

The configuration classes of the EEP parameters depend on the supported configuration variants. For their definitions please see the Eep_30_XXspi01_bswmd.arxml file.

6.2 Configuration in Data Base

The EEP does not provide any data base attribute.

6.3 Configuration of XML/ OIL Attributes

The EEP does not provide any XML/ OIL attribute.

6.4 Configuration with a GCE

The EEP can be easily configured with a GCE. The BSWMD containers and parameters (config class, type, range, etc.) can be visualized in a BSWMD editor or directly in GCE. In order to understand the purpose of every container and parameter please check the appropriate description fields.

The following sub-chapters describe some EEP configuration generalities of which the user has to be aware during component configuration.

6.4.1 EEP Published Information

The purpose of this EEP is to support devices that are behaviourally compatible with Microchip 25LC1024. As the container `EepPublishedInformation` publishes values that are device specific, it means the component would support only one device. Consequently the content of `EepPublishedInformation` is partially ignored and the parameters that vary from device to device shall be retrieved from container `EepDeviceInformation` (which is Vector specific and has the multiplicity 1..n).

In order to see which `EepPublishedInformation` parameters are relevant and which are ignored please check the appropriate description fields.

6.4.2 EEP Device Pre-Configuration and Configuration

Some specific EEPROM chips (e.g. Microchip 25LC1024), explicitly validated by Vector are already pre-configured and can be found in Eep_30_XXspi01_pre_bswmd.arxml. If the application needs one EEPROM which is pre-configured, the user will only have to assign the pre-configured container to the BSWMD parameter `EepDeviceInformationRef`.

If the desired EEPROM is not pre-configured, the user will have to create a new instance of container `EepDeviceInformation` and to input manually the chip properties (e.g. page size, write time, etc.); afterwards the user will assign this new created container to the BSWMD parameter `EepDeviceInformationRef`.

6.4.3 ASR3 Compatibility

Even though this EEP is released according ASR4, it can be used also in ASR3 projects/ environments. The ASR4 BSWMD file is compatible with ASR3 BSWMD file, except:

> there is new ASR4 container that refers the DEM error codes (see the ASR4 BSWMD container `EepDemEventParameterRefs`)

> the ASR3 BSWMD parameter `SpiReference` was renamed (see the ASR4 BSWMD parameter `EepSpiReference`)

Due to a general change in ASR4 BSWMD file structure (there is an extra `EcuDefs` package in every component, so not only in EEP) the references that point to AUTOSAR standard components cannot be resolved any longer.

Consequently the BSWMD container `EepDemEventParameterRefs` shall not be instantiated. In addition to that, the references BSWMD parameters `EepSpiReference`/ `EepDioReference`[1] shall remain empty. In this case the components DEM and SPI will have to generate the following fixed symbolic names:

- `EEP_E_CANCEL_FAILED` – DEM event if a job cancelation failed
- `EEP_E_READ_FAILED` – DEM event if read job failed
- `EEP_E_COMPARE_FAILED` – DEM event if compare job failed
- `EEP_E_TEST_COM_FAILED`[2] – DEM event if test communication job failed
- `EEP_E_WRITE_FAILED` – DEM event if write job failed
- `EEP_E_ERASE_FAILED` – DEM event if erase job failed
- `EEP_E_SET_LOCK_LEVEL_FAILED`[3] – DEM event if unlock job failed (1st instance)
- `EepCommandDataSequence` – SPI command and data sequence (1st instance)
- `EepCommandDataCommandChannel` – SPI command channel (1st instance)
- `EepCommandDataDataChannel` – SPI data channel (1st instance)
- `EepCommandOnlySequence` – SPI command only sequence (1st instance)
- `EepCommandOnlyCommandChannel` – SPI command only channel (1st instance)
- `EepCommandDataSequence_Inst2` – SPI command and data sequence (2nd instance)
- `EepCommandDataCommandChannel_Inst2` – SPI command channel (2nd instance)
- `EepCommandDataDataChannel_Inst2` – SPI data channel (2nd instance)
- `EepCommandOnlySequence_Inst2` – SPI command only sequence (2nd instance)
- `EepCommandOnlyCommandChannel_Inst2` – SPI command only channel (2nd instance)

[1] The parameter `EepDioReference` is allowed to remain empty also when the references can be resolved, meaning that the EEPROM write protection pin is not used by the hardware design (see EEPROM software protection mode). As a consequence of this feature, the EEPROM write protection pin cannot be used in case of ASR3 projects/ environments.

[2] The symbolic name `EEP_E_TEST_COM_FAILED` shall be generated only if the test communication API is enabled.

[3] The symbolic name `EEP_E_SET_LOCK_LEVEL_FAILED` shall be generated only if the unlock API is enabled.

Note
If any of the above mentioned fixed symbolic names cannot be generated by DEM or SPI for any reason, there's still the possibility to include a wrapper for these definitions. E.g. the SPI generates `SpiConf_SpiChannel_EepCommandDataCommandChannel` instead of `EepCommandDataCommandChannel`, the user should add a wrapper include-file to EepCfgIncludeList in EepInitConfiguration, which contains an appropriate mapping, such as:

```
#include "Spi_Cfg.h"
#define EepCommandDataCommandChannel    \
SpiConf_SpiChannel_EepCommandDataCommandChannel
```

6.4.4 SPI Configuration

The EEP requires 2 separate SPI Sequences per instance, one for processing EEPROM instructions that contain data (e.g. WRITE) and one for processing EEPROM instructions that do not contain data (e.g. WREN). Special names are required only when EEP is used in ASR3 projects/ environments, for details please see the chapter 6.4.3.

6.4.4.1 SPI Command and Data Sequence

The SPI sequence used to process EEPROM instructions that contain a command and data has to adhere to the following settings:

- The sequence end notification (see BSWMD parameter `SpiSeqEndNotification`) has to be configured to call the EEP communication end notification described in chapter 5.5.1.
- The sequence references exactly one job – command and data job:
  - The job end notification (see BSWMD parameter `SpiJobEndNotification`) has to be disabled
  - First channel referenced by the job – command channel:
    - The channel has to be of type “external buffer” (see BSWMD parameter `SpiChannelType`)
    - The transmission word width has to be set to 8bit (see BSWMD parameter `SpiDataWidth`)
    - The maximum length has to be exactly the number of address bytes (for the chosen EEPROM) + 1 (see BSWMD parameter `SpiEbMaxLength`)
    - Transfer has to start with most significant bit first (see BSWMD parameter `SpiTransferStart`)
  - Second channel referenced by the job – data channel:
    - The channel has to be of type “external buffer” (see BSWMD parameter `SpiChannelType`)
    - The transmission word width has to be set to 8bit (see BSWMD parameter `SpiDataWidth`)
    - The maximum length has to be configured to match the maximum of EEPROM page size and EEP read size (see BSWMD parameter `SpiEbMaxLength`)
    - The transfer has to start with most significant bit first (see BSWMD parameter `SpiTransferStart`)
    - The default data has to be set to 0xFF (see BSWMD parameter `SpiDefaultData`)

Caution
Depending on SPI specific generator implementation, it may result that the channels order (priority) in the command and data job channels list is not as expected (namely: first must be the command channel and second must be the data channel). Please check the channels order (priority) and guarantee that this will meet the expectations.

6.4.4.2 SPI Command Only Sequence

The SPI sequence used to process EEPROM instructions that contain only a command has to adhere to the following settings:

- The sequence end notification (see BSWMD parameter `SpiSeqEndNotification`) has to be configured to call the EEP communication end notification described in chapter 5.5.1.
- The sequence references exactly one job – command only job:
  - The job end notification (see BSWMD parameter `SpiJobEndNotification`) has to be disabled
  - The channel referenced by the job – command only channel:
    - The channel has to be of type “external buffer” (see BSWMD parameter `SpiChannelType`)
    - The transmission word width has to be set to 8bit (see BSWMD parameter `SpiDataWidth`)
    - The maximum length has to be exactly 1 byte (see BSWMD parameter `SpiEbMaxLength`)
    - The transfer has to start with most significant bit first (see BSWMD parameter `SpiTransferStart`)

6.5 Configuration with GENy

The EEP is not supported by GENy tool.

6.6 Configuration with DaVinci CFG

The EEP is currently supported in DaVinci CFG (CFG4 and CFG5) tool, but does not provide a comfort editor. The configuration can be currently performed only in the GCE view of DaVinci CFG. For guidelines regarding configuration with a GCE please check chapter 6.4.

Independent whether the parameterization is performed via CFG4 or CFG5, the EEP module has only one generator - a CFG5 native generator.

6.7 Configuration with CANgen

The EEP is not supported by CANgen tool.

6.8 Configuration manually in Header Files

The EEP configuration cannot be adjusted by means of a user-config file that can be inserted in a header file.

7 Glossary and Abbreviations

7.1 Glossary

Term	Description
Data Block	A data block may contain 1..n bytes and is used within the API of the EEPROM driver. Data blocks are passed with: - Address offset in EEPROM - Pointer to memory location - Length to the EEPROM driver.
Data Unit	The smallest data entity in EEPROM. The entities may differ for read/ write/ erase operation. Example 1: MCS12(X) Read: 1 byte Write: 2 bytes Erase: 4 bytes Example 2: external SPI EEPROM device Read/ Write/ Erase: 1 byte
DaVinci CFG	Configurator/ Embedded Architecture Designer - configuration and generation tool for MICROSAR components
EEP Slow Mode	Some external SPI EEPROM devices provide the possibility of different access modes. In slow (normal) mode the data exchange with the EEPROM device via SPI is performed byte wise. This allows for cooperative SPI usage together with other SPI devices like I/O ASICs, external watchdogs etc.
EEP Fast Mode	Some external SPI EEPROM devices provide the possibility of different access modes. In burst mode the data exchange with the EEPROM device via SPI is performed block wise. The block size depends on the EEPROM properties, an example is 64 bytes. Because large blocks are transferred, the SPI is blocked by the EEPROM access in burst mode. This mode is used during ECU start-up and shut-down phases where fast reading/ writing of data is required.
EEP Cell	Smallest physical unit of an EEPROM device that holds the data. Usually 1 byte.
Incremental Register	Some EEPROMs provide in some pages (usually the pages at the beginning of the EEPROM) incremental registers pages. These pages are organized as words. The initial content of each word on this page is 0000h. When writing to a word, a logic comparator verifies that the new value is larger than the value currently stored. If the new value is equal to or smaller than the current one, no operation is performed. It is impossible to write a value lower than the previous one!

Table 7-1 Glossary

7.2 Abbreviations

Abbreviation	Description
API	Application Programming Interface
AUTOSAR	Automotive Open System Architecture
BSW	Basis Software
BSWM	Basis Software Mode Manager
DEM	Diagnostic Event Manager
DET	Development Error Tracer
DIO	Digital Input/ Output
EA	EEPROM Abstraction
ECU	Electronic Control Unit
HIS	Hersteller Initiative Software
ISR	Interrupt Service Routine
MEMIF	Memory Interface
MICROSAR	Microcontroller Open System Architecture (the Vector AUTOSAR solution)
RTE	Runtime Environment
SPI	Serial Peripheral Interface
SRS	Software Requirement Specification
SWC	Software Component
SWS	Software Specification

Table 7-2 Abbreviations

8 Contact

Visit our website for more information on

- News
- Products
- Demo software
- Support
- Training data
- Addresses

www.vector-informatik.com